

Building Global IT Professionals **since 2008**

19K+

Certified Students

160+

Courses

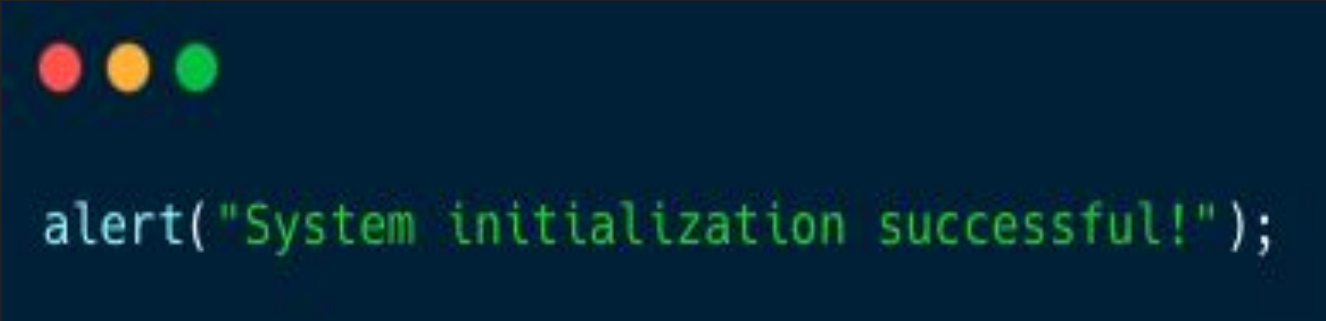
85+

Qualified Teachers

www.broadwayinfosys.com

JavaScript

- Interpreted Architecture: Code runs natively on the client browser instantly without pre-compilation.
- Functional Reflexes: Methods execute immediately in response to actions like key clicks or inputs.
- Full Stack Utility: Capable of managing client-side behavior as well as backend database layers.



```
alert("System initialization successful!");
```

Linking JavaScript to HTML

Connecting a JavaScript file to an HTML document so JavaScript can make the webpage interactive.

Modular Isolation: Separating logic ensures clean, reusable, and easy-to-read code architectures.

Optimized Load-order: Put code tags right above the body close tag to prevent loading delays.

Defer Script Handling: Using modern defer tags ensures standard document styling parses cleanly.



```
<!-- Anchor external scripts cleanly -->
<script src="app.js" defer></script>

<!-- Inline emergency diagnostic hooks -->
<script>
    console.log("Dynamic inline parsed!");
</script>|
```



Variables: let, const, & var

- **const:** Protects variables from unexpected reassignment errors downstream.
- **let:** Allows variable contents to safely scale and update inside tracking operations.
- **var:** Highly deprecated due to outdated scope hoisting and memory leak risks.

```
// Const guarantees value preservation
const TAX_RATE = 0.13;

// Let lets storage values mutate
let cartTotal = 150;
cartTotal = 180; // safely overwritten

// Avoid legacy var declaration
var legacyStr = "Deprecated Scope";
```



Data Types & Classification

Data Type	Category	Description	Example
String	Primitive	Stores text values	<code>"Hello"</code>
Number	Primitive	Stores integers and decimal numbers	<code>25</code> , <code>3.14</code>
Array	Non-Primitive	Stores an ordered collection of values	<code>["Apple", "Banana"]</code>
Object	Non-Primitive	Stores data as key-value pairs	<code>{ name: "John", age: 25 }</code>
Function	Non-Primitive	Stores reusable blocks of code	<code>function greet() {}</code>



What are Functions?

- A function is a reusable block of code that performs a specific task.
- Helps reduce code duplication and improve readability.

Type	Syntax
Function Declaration	function greet() {}
Function Expression	const greet = function() {};
Arrow Function (ES6) ★	const greet = () => {};

```
function greet() {  
  console.log("Hello!");  
}  
  
greet();
```



Arrow Functions (ES6)

- Shorter syntax
- Implicit return for single expressions
- Commonly used with array methods (map, filter, reduce)



```
function add(a, b) {  
  return a + b;  
}
```



```
const add = (a, b) => {  
  return a + b;  
};
```



Destructuring Arrays & Objects (ES6)

Destructuring is an ES6 feature that lets you extract values from arrays or properties from objects into separate variables.



```
const colors = ["Red", "Green", "Blue"];

const [first, second, third] = colors;

console.log(first); // Red
console.log(second); // Green
```



```
const person = {
  name: "John",
  age: 25,
  city: "London"
};

const { name, age } = person;

console.log(name); // John
console.log(age); // 25
```



DOM Elements

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects. JavaScript uses the DOM to access and modify HTML elements.

Method	Description	Example
<code>getElementById()</code>	Selects an element by its ID	<code>document.getElementById("title")</code>
<code>getElementsByClassName()</code>	Selects elements by class name	<code>document.getElementsByClassName("box")</code>
<code>getElementsByTagName()</code>	Selects elements by tag name	<code>document.getElementsByTagName("p")</code>
<code>querySelector()</code>	Selects the first matching CSS selector	<code>document.querySelector(".box")</code>
<code>querySelectorAll()</code>	Selects all matching CSS selectors	<code>document.querySelectorAll(".box")</code>





```
<p id="demo"></p>
```

```
<script>
```

```
// Access a paragraph Element
```

```
const myPara = document.getElementById("demo");
```

```
// Change the content of the Element
```

```
myPara.innerHTML = "Hello World!";
```

```
</script>
```



Common DOM Properties & Methods

Property / Method	Purpose
<code>textContent</code>	Change or get text
<code>innerHTML</code>	Change HTML content
<code>style</code>	Modify CSS styles
<code>classList.add()</code>	Add a CSS class
<code>classList.remove()</code>	Remove a CSS class
<code>setAttribute()</code>	Set an HTML attribute





```
// Swap textual tags  
heading.textContent = "Hello, Semester 2!";  
  
// Modify styling instantly  
activeButton.style.backgroundColor = "#10b981";  
  
// Add class rules cleanly  
heading.classList.add("text-glow");
```



Event Handlers & Triggers

- What are Events?

Events are actions that occur in the browser, such as a user clicking a button, typing in a textbox, or moving the mouse.

- What is an Event Handler?

An event handler is a JavaScript function that runs automatically when an event is triggered.

```
activeButton.addEventListener("click", () => {  
    console.log("Action click fired!");  
});  
  
// Listen for dynamic input changes  
inputBox.addEventListener("input", (e) => {  
    console.log(e.target.value);  
});
```



Event	Trigger	Example
click	User clicks an element	Button click
input	User types in an input field	Search box
change	Input value changes	Select dropdown
submit	Form is submitted	Login form
load	Page finishes loading	Webpage loaded



THANK YOU!



Shree Ganesh Marg, Subidhanagar, Tinkune, Kathmandu

01-4117578 / 411849 / 9841002000

www.broadwayinfosys.com